

Schedule-Free On-Demand Water Transport in Bergen: Progress Report

September 4, 2026

Abstract

This note documents the progress made so far on a project that designs the real-time dispatch logic for a small-boat, on-demand water transport service in Bergen, Norway. The service does not exist yet: rather than optimizing an existing system, the project builds a model of how such a service would operate, replacing fixed timetables with a dispatch policy that recomputes itself every few minutes. Five pieces are covered in this report: a navigable routing module that computes real, coastline-respecting travel times between the four candidate stops (replacing an earlier straight-line approximation); a synthetic demand model anchored in official Norwegian population and employment data; a first, deliberately simple simulation environment, implemented to the Gymnasium interface standard, that hosts the dispatch policy; a fixed, rule-based reference policy verified against that environment at three increasing scales; and an initial reinforcement-learning agent, trained with proximal policy optimization, intended to eventually replace that fixed rule. The agent does not yet match the reference policy, and this report documents both that gap and the diagnostic process already carried out to narrow it. This is a partial, working progress report, not a finished piece of work.

Contents

1	Introduction and Motivation	3
2	Related Work	3
3	Navigable Routing	3
4	Demand Model	5
4.1	Catchment areas and masses	5
4.2	From population and employment to origin-destination intensity	6
4.3	From intensity to Poisson arrival rates	7
4.4	Arrivals and group sizes	8
4.5	Persons vs. groups	9
5	Simulation Environment	9
5.1	State	9
5.2	Actions	10
5.3	Transition	10
5.4	Reward	10
6	Reference Dispatch Policy	10
6.1	Decision rule	10
6.2	Coordinating several free boats at once	11
6.3	Verification at three scales	11

6.4	Known limitations	15
7	Reinforcement Learning Agent	15
7.1	Method	15
7.2	Implementation, observation, and action	15
7.3	Reward function	16
7.4	Fresh demand on every reset	17
7.5	Diagnosing and fixing a degenerate first training run	17
7.6	Training run and comparison against the reference policy	19
7.7	Discussion and next step	22

1 Introduction and Motivation

Bergen is a coastal city whose public transport network is structured as a hub-and-spoke system centered on the city center, a layout imposed by the surrounding fjords and mountains. As the city grows and tourism increases, this network is under increasing pressure, and land-based expansion is constrained by the same geography that makes it congested in the first place. The fjords that divide the city are, at the same time, an underused transport corridor.

This project explores a small-boat, on-demand water transport service as a complementary mode. Two principles guide its design. First, the service is not modeled as a profit-making venture (public transport of this kind is rarely self-financing, and its justification is environmental and related to congestion relief, not revenue); the objective is to deliver a good level of service efficiently, not to maximize income. Second, the flexible layer of the service has no fixed timetable: a passenger requests a trip and a boat is dispatched to serve it, subject to a waiting-time guarantee, and the dispatch policy is recomputed at short, regular intervals rather than following a pre-published schedule. Section 2 situates this "schedule-free" operating philosophy, and the static water-taxi model it builds on, within the closely related literature.

Scope of this report. This is a progress note, not a finished piece of work. It covers the routing and demand modeling that feed the project (Sections 3 and 4), a first version of the simulation environment (Section 5), a fixed reference dispatch policy verified against that environment (Section 6), and a first attempt at replacing that policy with a trained reinforcement-learning agent (Section 7). The environment is intentionally simple: it is a first iteration meant to establish a working structure, not the final model the project targets, and both policies described here are early results rather than settled conclusions. The reinforcement-learning agent in particular has not yet matched the reference policy it is meant to eventually surpass; Section 7 reports that gap honestly, together with the diagnostic work already done to close it.

2 Related Work

Three papers anchor the modeling choices described in this report.

Gu and Wallace [2021] formulate a static location, fleet-sizing, and routing model for a water-taxi system connecting the same four stops used in this project (Kleppestø, Laksevåg, Bryggen, and Sandviken). It is the closest precedent in terms of the physical instance, and it is used in Section 4 as one reference point for the scale of daily demand.

Braathen et al. [2024] study autonomous ferry scheduling under labor regulations, comparing autonomous and manually operated fleets. The demand-generation approach in Section 4 is adapted from their multiplicative, gravity-style formulation, and the group-size statistics they report are used to calibrate the group-size distribution used here.

Sánchez-Martínez et al. [2016] develop a schedule-free operating paradigm for high-frequency transit control. Their central point, that abandoning fixed schedules in favor of real-time, resource-aware control can improve service quality provided the underlying control problem can be solved effectively, is the conceptual starting point for treating dispatch in this project as a control problem rather than a scheduling problem.

3 Navigable Routing

Travel times between the four stops are computed from a navigable-routing module rather than from straight-line (Haversine) distances. An earlier, straight-line version of the travel-time matrix was found, on visual inspection, to draw lines that cross land near Bryggen, which sits inside the Vågen bay next to the Nordnes peninsula (straight-line distances are therefore not usable directly and are corrected here).

Water/land classification. A basemap tile mosaic covering the four stops is downloaded at zoom level 14, which corresponds to a fixed Web Mercator resolution of $156543.03392/2^{14} \approx 9.55$ m/pixel. Because Web Mercator distorts distances by a factor of $1/\cos(\text{latitude})$, and Bergen sits at approximately 60.4°N , the real ground resolution is $9.55 \times \cos(60.4^\circ) \approx 4.7$ m/pixel. Each pixel is classified as water or land by its Euclidean distance, in RGB space, to a reference water color sampled from the basemap style (threshold of 15 color units); the resulting mask is pixel-for-pixel aligned with the source map, since both are produced by the same Web Mercator projection formula and the same Earth radius.

Graph and shortest paths. Each water pixel becomes a node in an 8-connected graph (4 orthogonal and 4 diagonal neighbors), with edge weights equal to the real distance between pixel centers; land pixels are simply absent from the graph, so no edge can cross land. The four stops are snapped to the nearest water pixel, restricted to the graph’s main connected component, to avoid anchoring a stop to an isolated puddle unconnected to open water. A multi-source Dijkstra search (`scipy.sparse.csgraph.dijkstra`) then computes the shortest navigable path between every pair of stops in a single call, exact because all edge weights are non-negative real distances.

Design speed and resulting matrix. Rather than calibrating vessel speed against an existing ferry line, a fixed design speed of 30 km/h (≈ 16.2 knots) is used for the whole fleet, treated as a design decision for the small on-demand vessels rather than an empirical calibration. Table 1 reports the resulting travel-time matrix, and Table 2 compares it against the straight-line matrix it replaces: routing around land adds between 6% and 24% to travel distance depending on the pair, with Laksevåg (whose corrected location sits further inside the Puddefjorden) showing the largest correction because the detour around the Nordnes peninsula is large relative to its own distance to the other stops.

Table 1: Navigable travel-time matrix (minutes), 30 km/h design speed.

	Kleppestø	Laksevåg	Bryggen	Sandviken
Kleppestø	-	5.3	11.5	9.6
Laksevåg	5.3	-	8.6	8.6
Bryggen	11.5	8.6	-	6.8
Sandviken	9.6	8.6	6.8	-

Table 2: Straight-line vs. navigable travel times, all six stop pairs.

Pair	Straight-line (min)	Navigable (min)	Difference
Laksevåg ↔ Bryggen	7.0	8.6	+23.8%
Kleppestø ↔ Sandviken	8.7	9.6	+10.5%
Bryggen ↔ Sandviken	6.3	6.8	+8.9%
Kleppestø ↔ Laksevåg	4.9	5.3	+7.8%
Kleppestø ↔ Bryggen	10.7	11.5	+7.3%
Laksevåg ↔ Sandviken	8.1	8.6	+6.4%

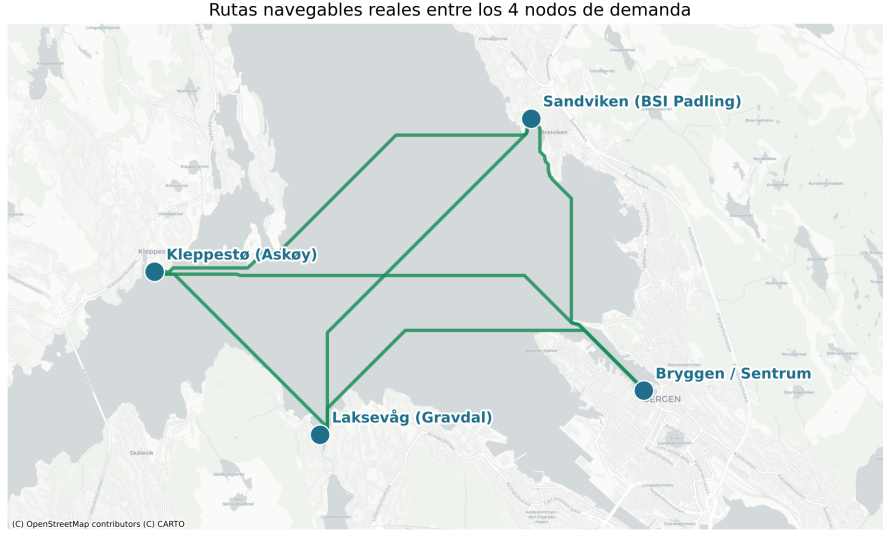


Figure 1: The six navigable routes between the four demand stops, computed by Dijkstra search over the water-classified pixel graph.

The known limitation of this method is that an 8-connected grid only offers eight travel directions per step, so a shortest path can zig-zag slightly instead of following a perfect diagonal when the true direction falls between two of the eight allowed ones; the resulting overcost is a few percent in the worst case and does not affect the central conclusion that the method reliably avoids land.

4 Demand Model

No real demand data exists for this service, since it does not operate yet. Demand is therefore synthesized, but anchored in real open data wherever possible.

4.1 Catchment areas and masses

Population and employment are read from official Statistics Norway (SSB) 250-metre grids. For each of the four stops, a catchment area was hand-drawn by the project team in QGIS; every *grunnkrets* (Norway’s smallest statistical reporting unit) that the drawn area touches, even partially, is included in full for that stop, and duplicate assignments at shared boundaries are resolved by keeping each grunnkrets with the stop it overlaps the most. Population and employment are then summed over the SSB grid cells inside the resulting area. Table 3 reports the resulting masses, and Figure 2 shows which grunnkretser were assigned to each stop.

Table 3: Catchment area, population, and employment by node (SSB grids, 2026).

Node	Grunnkretser	Area (km ²)	Population	Employment
Kleppestø (Askøy)	28	80.6	25,153	7,550
Laksevåg (Gravdal)	18	14.9	15,986	7,760
Bryggen / Sentrum	67	20.2	27,695	52,213
Sandviken (BSI Padling)	21	14.6	11,109	4,106

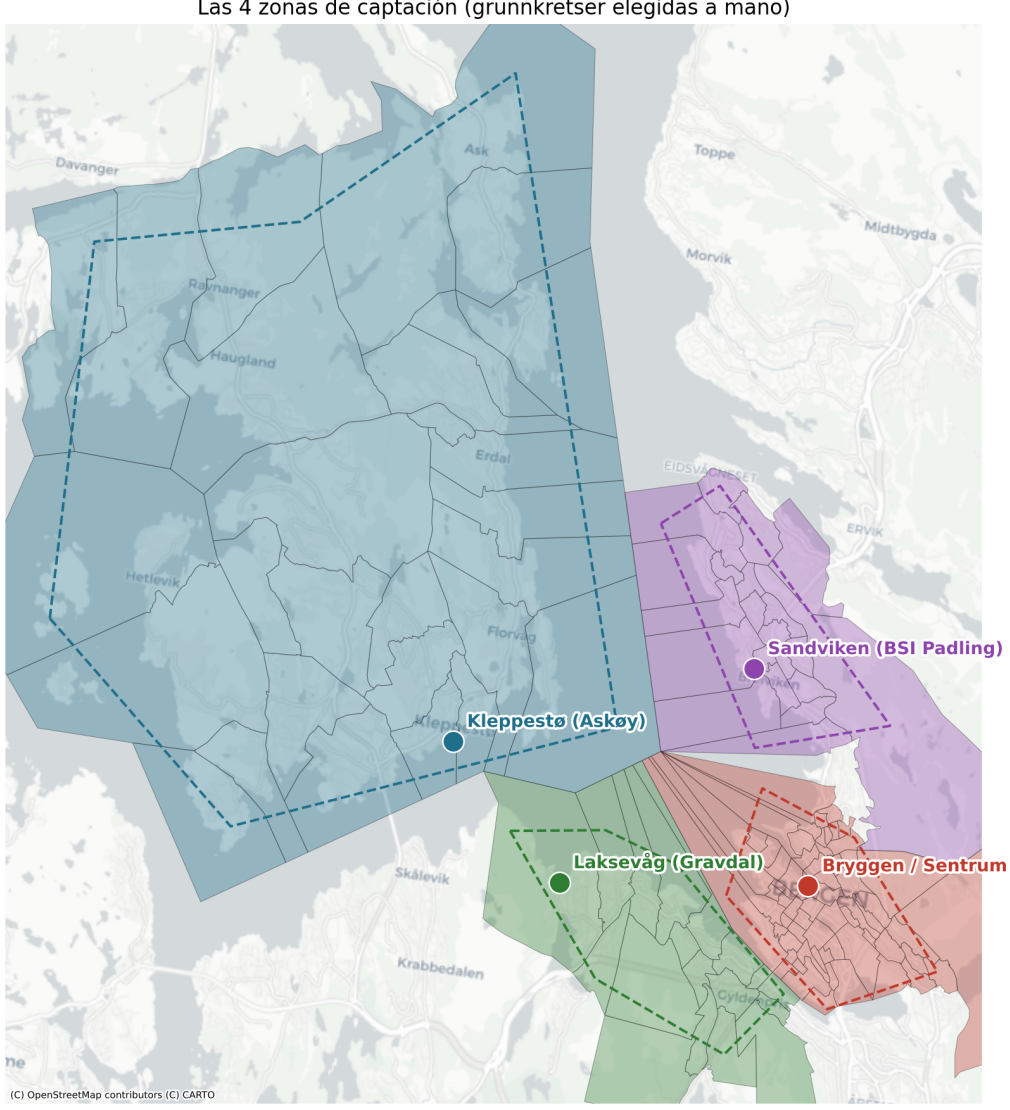


Figure 2: The four hand-drawn catchment areas (dashed outline) and the grunnkretser assigned to each node (filled).

4.2 From population and employment to origin-destination intensity

Braathen et al. [2024] generate synthetic demand using a multiplicative popularity score that combines different factors associated with the attractiveness of stations, travel times, weekdays, and hours. This project adopts the general logic of using a score to represent the relative attractiveness or intensity of demand, but replaces the judgment-based popularity factors with observable population, employment, and university-related measures for the Bergen study area. The score used in this project is defined as follows.

Let $\text{pop}(n)$ and $\text{emp}(n)$ be the population and employment of node n from Table 3, and let $u(n) \in [0, 1]$ be the university weighting factor described below. Masses are first normalized against the largest node, so that both scales end up comparable before being multiplied:

$$\text{pop}_{\text{norm}}(n) = \frac{\text{pop}(n)}{\max_m \text{pop}(m)}, \quad \text{emp}_{\text{norm}}(n) = \frac{\text{emp}(n)}{\max_m \text{emp}(m)} \cdot u(n). \quad (1)$$

$u(n)$ compensates for the fact that registered employment does not count students, and Bergen is a university city; no open source of fine-grained enrollment data was found, so $u(n)$ is fixed

by hand for now (1.0 at Bryggen and Sandviken, reduced at Kleppestø and Laksevåg), declared here as a design choice rather than measured data. A third, "mixed" mass averages the two, used at times of day with no clear commute direction:

$$\text{mix}(n) = \frac{1}{2}(\text{pop}_{\text{norm}}(n) + \text{emp}_{\text{norm}}(n)). \quad (2)$$

Each of the four time bands in Table 4 assigns a role (population, employment, or mixed) to the origin side and to the destination side of every trip; write $w(n, \text{role})$ for whichever of the three masses above that role selects. The origin-destination intensity for band b and the ordered pair (o, d) , $o \neq d$, is then

$$I_b(o, d) = w(o, \text{role}_{\text{origin}}(b)) \cdot w(d, \text{role}_{\text{destination}}(b)) \cdot f_{\text{dist}}(o, d) \cdot v(b), \quad (3)$$

where $v(b)$ is the band's relative volume factor (Table 4) and f_{dist} is a distance-decay term not present in Braathen et al. [2024]'s formulation, added in this project:

$$f_{\text{dist}}(o, d) = \exp\left(-\frac{t(o, d)}{\tau}\right), \quad \tau = 15 \text{ min}, \quad (4)$$

with $t(o, d)$ the navigable travel time from Table 1. $I_b(o, d)$ has no unit of its own; it is a relative weight, only meaningful compared to the other 11 pairs of the same band, and is turned into an actual passenger count in the next step.

Table 4: Time bands, relative demand volume, and origin/destination roles.

Band b	Hours	$v(b)$	Role _{origin} / Role _{destination}
Morning	06-09	1.00	population / employment
Midday	09-15	0.40	mixed / mixed
Afternoon	15-18	1.00	employment / population
Evening	18-24	0.15	mixed / mixed

4.3 From intensity to Poisson arrival rates

Every hour of the day inherits the intensity of the band it belongs to (a step function across bands, not a smoothed hourly curve): $I_h(o, d) := I_{b(h)}(o, d)$ for hour h in band $b(h)$. This is computed for all 24 operating hours and all 12 ordered pairs, covering all four bands, not a subset.

The day's total passenger budget is set as a percentage of the population of the four catchment zones of Table 3 (not the total population of Bergen and Askøy), scaled down on weekends:

$$S = p \cdot P_{\text{zonas}} \cdot f_{\text{semana}}, \quad P_{\text{zonas}} = \sum_n \text{pop}(n), \quad (5)$$

where p is `demanda.porcentaje_poblacion_dia` in the demand model's configuration file (currently 10%, an explicit, changeable team parameter, not a value taken from either reference paper), $P_{\text{zonas}} = 79,943$ is the sum of the population column of Table 3, and f_{semana} is 1.0 on weekdays and 0.4 on weekends. S is a number of *passengers*; dividing by the mean group size $\bar{g} = 8.6$ (Section 4.4) gives the day's total number of *groups*, $G = S/\bar{g}$, which is then distributed across all hour-pair cells in proportion to their share of the day's total intensity:

$$\lambda(o, d, h) = \frac{I_h(o, d)}{\sum_{h', o', d'} I_{h'}(o', d')} \cdot G. \quad (6)$$

Within each hour, the number of arriving groups for pair (o, d) is drawn $N \sim \text{Poisson}(\lambda(o, d, h))$, and each group's arrival minute is drawn uniformly within that hour.

As a reference point for the order of magnitude of S : Gu and Wallace [2021], who study the same four stops, assume 300 travel requests per day in their case study. That number is for a water-taxi service, a different kind of offering (booked, point-to-point, presumably priced and scheduled differently from a public-transport-style service), so it is not directly comparable to S here; it is mentioned only because it is a verifiable, same-instance data point, not as a validation of the 10% figure.

4.4 Arrivals and group sizes

Passenger arrivals within each hour follow the Poisson process described above. Braathen et al. [2024] report a mean group size of 8.6 passengers with standard deviation 2.6, but do not specify a sampling distribution for it (their own group size is a deterministic function of the popularity score pop_{pg} , not a random draw). Since group size is a count of people, a discrete, non-negative distribution is used instead of a continuous one: a $\text{Binomial}(n = 40, p = 0.215)$, calibrated by matching moments to Braathen et al.’s reported mean and standard deviation, which reproduces the target mean exactly (8.60) and the target standard deviation closely (2.598 against 2.6).

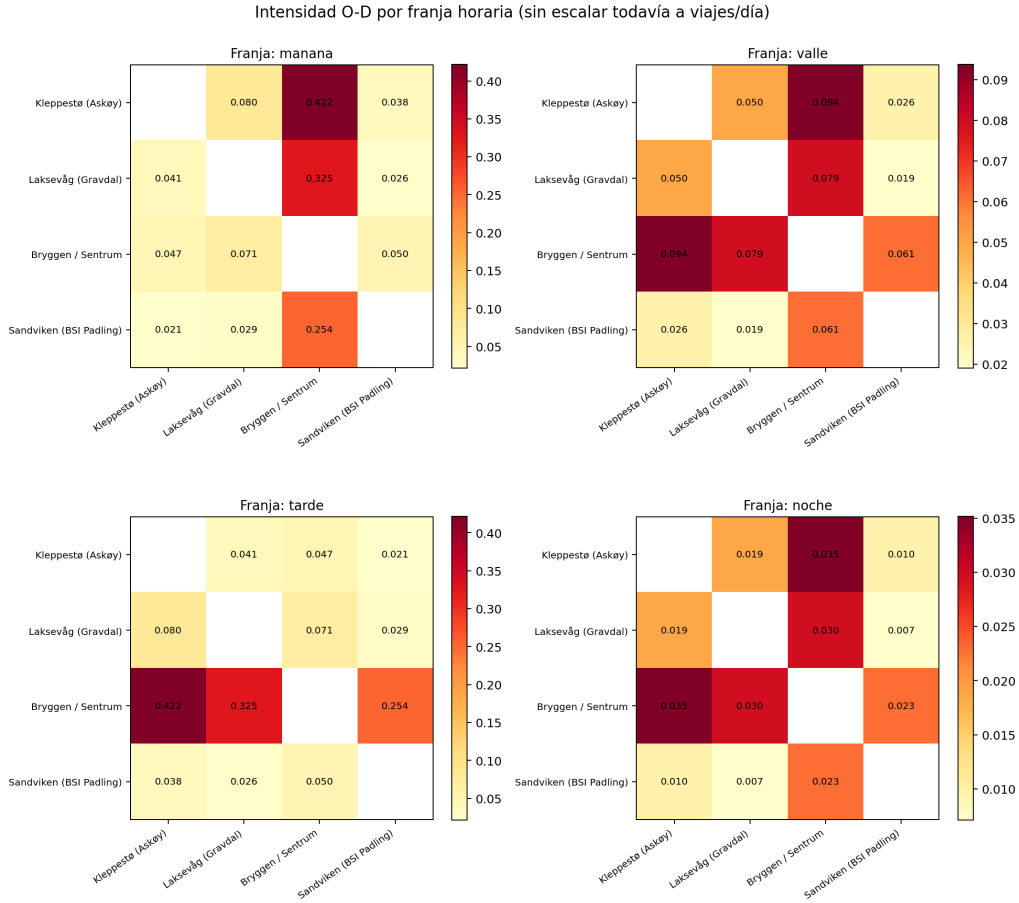


Figure 3: Origin-destination intensity $I_b(o, d)$ by time band. Morning traffic concentrates toward Bryggen; the afternoon band reverses the pattern.

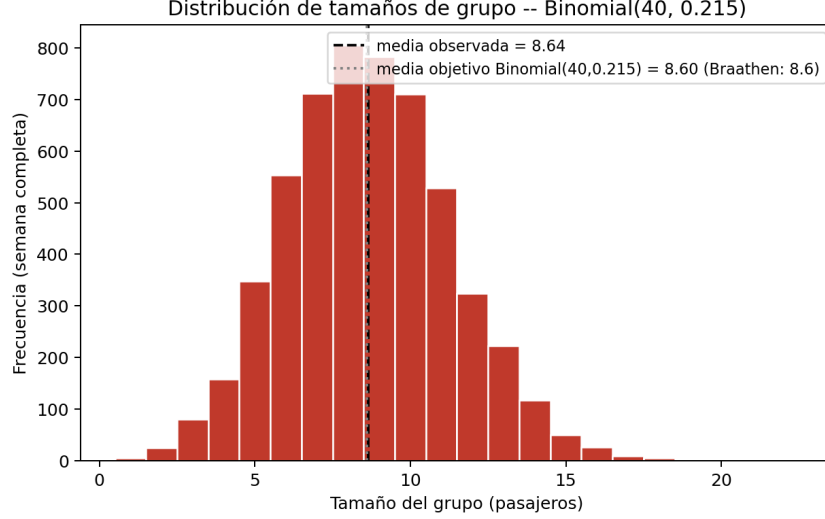


Figure 4: Observed group-size distribution over a simulated week, against the target Binomial(40, 0.215).

4.5 Persons vs. groups

A group, as generated above, is the atomic unit of the demand model. For the dispatch simulation environment (Section 5), each group can optionally be exploded into individual persons, each inheriting the same origin, destination, and arrival time as the group they came from; this changes the atomic unit seen by the simulator’s capacity and boarding logic without altering when or where demand appears.

5 Simulation Environment

This is a first, simple iteration. The environment described in this section establishes a working structure for the dispatch problem, not the final model the project targets. It is deliberately minimal: four stops, direct point-to-point boat trips, and a small set of rules, chosen so that the environment itself (its mechanics, its bookkeeping, its interface) could be verified before adding any complexity to the model of how boats and passengers behave. Extensions are expected as the project continues.

The dispatch problem is modeled as a Markov decision process and implemented as a discrete-time simulation environment following the Gymnasium interface standard (`reset()` / `step()`), which is not a library of pre-built simulators but an interface contract: the environment implements the entire boat-and-passenger logic itself, and only its shape is standardized, so that a reinforcement-learning library can later drive it without modification. The environment advances in fixed 2-minute steps, rather than jumping between discrete events, for simplicity and because it maps directly onto the `step()` call.

5.1 State

The state has three parts. For each boat: origin node, destination node (equal to the origin if idle), minutes remaining until arrival, current occupancy, and whether it is free to receive a new order. For demand: one queue per origin-destination pair (twelve pairs for four nodes, not one queue per node), holding the number of people waiting and the waiting time of the oldest person in that specific pair. For time: minute of day and day of week. This rich, dictionary-shaped state is used for logging and inspection; a separate fixed-size numeric vector is derived from it for the eventual learning agent, using one-hot node encoding and cyclical (sine/cosine) time encoding

so that midnight and day-of-week wraparound do not appear as artificial discontinuities to a neural network.

5.2 Actions

Every 2 minutes, each free boat receives one of five orders: travel to one of the four nodes, or wait. A boat already en route ignores any new order and continues to its destination; it is not redirected mid-trip. Boarding follows a fixed rule, not chosen by the policy: when a free boat at node A is ordered to travel to node B , it boards strictly from the queue for the pair (A, B) , in arrival order, up to capacity, and travels directly to B , where everyone aboard disembarks. Trips are therefore always direct, point-to-point journeys between exactly two stops; a boat never carries passengers with mixed destinations, since it can only ever board from the one queue that matches the trip it was just ordered to make.

5.3 Transition

Each step executes, in order: (1) apply the chosen action to every free boat, boarding from the matching pair queue as described above; (2) advance all boats in transit by the fixed time step, disembarking everyone aboard any boat that reaches its destination; (3) add to the queues any new arrivals whose arrival time falls within the interval just elapsed (these do not board the boat that just departed in step 1, since they had not yet arrived when boarding happened); (4) compute the step’s reward and assemble the new state.

5.4 Reward

The reward returned at every 2-minute step is a single scalar, negative unless passengers are actually delivered, that penalizes letting people wait or travel beyond a tolerance and, more lightly, sending a boat under way; a positive term rewarding deliveries directly is added later for training the reinforcement-learning agent. This scalar plays no role in the reference dispatch policy of Section 6, which follows a fixed rule and never consults it; it exists purely to give the environment a well-defined objective for a policy that has to learn from experience. Because the reward is meaningful chiefly in that training context, and because it changed materially once patience was removed and again while diagnosing the training agent, its full definition and the reasoning behind each term are given together in Section 7.3, next to the training results it is meant to explain.

6 Reference Dispatch Policy

Before any policy is asked to learn, it is useful to have a simple, fixed rule that can be verified by hand and used as a point of comparison. This section describes that rule, a coordination problem that surfaced only once several boats used it together, and the results of running it on three test instances of increasing size.

6.1 Decision rule

The reference policy, referred to here as nearest-available, decides one free boat at a time and follows a single principle: serve the demand waiting at the boat’s own current stop before ever considering a trip to reposition toward demand elsewhere. Concretely, when boat b is free at node A , the policy first looks at the twelve pair queues (Section 5.1) restricted to those leaving A ; if any of them holds someone waiting, the boat is sent to the destination whose queue has been waiting longest, without regard to how many people are in it. Only when no one at all is waiting to leave A does the policy consider repositioning: it looks across every other pair queue

in the system and sends the boat, empty, toward the origin node of whichever queue has waited longest, so that the boat can pick that demand up once it arrives.

This rule is deliberately myopic. It never looks two moves ahead: a boat ordered to reposition simply travels to the node with the oldest waiting demand and re-evaluates from scratch once it gets there, rather than planning the pickup and the eventual delivery as a single two-leg trip. It is also strictly local-first, which is a real design choice with a real cost. Consider a boat that becomes free at Bryggen just as five people who arrived a minute earlier ask to go to Laksevåg, while a single person has already been waiting twenty minutes at Kleppestø for a boat to Sandviken. Nearest-available sends the boat to the five people at Bryggen, not to the person who has waited far longer at Kleppestø, because it never compares urgency across nodes while there is still anyone to serve locally. This keeps the rule simple to state and to verify, and it avoids abandoning demand the boat could serve immediately in favor of chasing something that still requires travel time; the cost is that a lightly connected node can, in principle, be left waiting for a long time if the busier nodes never run dry. Because the environment no longer removes anyone from a queue for having waited too long (Section 5.3), this is not a theoretical risk that resolves itself: nothing in the environment stops that wait from growing arbitrarily long if the policy never gets around to it. Whether this actually happens in practice is exactly what the verification below is designed to check.

6.2 Coordinating several free boats at once

A second issue appears only when more than one boat is free at the same node in the same step. Applying the decision rule above independently to each boat, using the same snapshot of the world for both, can make two boats choose the same destination: each one sees, say, eleven people waiting at a shared destination and decides that is the most urgent trip to make, without knowing the other boat just made the identical decision. The first boat to be processed picks up all eleven; the second arrives at the same destination with nobody left to serve, having paid the cost of the trip for nothing. This was not a hypothetical concern. It was found by inspecting the state of a verification run at a single simulated minute and confirmed in the accompanying event log, and it visibly hurt the fraction of demand served in that run.

The fix, applied whenever more than one boat is free in the same step, is to assign boats to actions one at a time rather than all at once from the same snapshot. Boats are processed in a fixed order; after each one is assigned a destination, the pair queue it would draw from is reduced locally, in a scratch copy of the state, by exactly what that boat would pick up, following the same first-in-first-out and capacity rule the environment itself uses when boarding. The next boat in line then applies the decision rule to this already-discounted picture of demand, so a second boat heading to the same busy destination sees only what would genuinely still be waiting for it, not what the first boat already claimed. This coordination step does not change the underlying decision rule; it only prevents two boats from acting on the same stale piece of information. All results reported below use this coordinated version, referred to as *fleet assignment*, never the single-boat rule applied independently to a group.

6.3 Verification at three scales

The coordinated policy was run on three test instances built from the demand model of Section 4, each scaled down from the model's own 10% daily participation rate so that a run stays easy to follow while still exercising the full mechanics of the environment. Table 5 summarizes the three.

Table 5: The three verification instances: scale, fleet, and purpose.

	Step 1	Step 2	Step 3
Time window	Morning, 06:00 to 09:00	Full day, 06:00 to 24:00	Full week (7 days)
Boats	2	3	3
Groups / people	23 / 202	133 / 1106	620 / 5429
2-minute steps	90	540	$7 \times$ up to 540
Purpose	Step-by-step visual check	Aggregate metrics, one day	Weekday/weekend pattern

Step 3 is run as seven independent one-day episodes, one per weekday, rather than as a single continuous week-long clock; the demand generator already applies its own weekday/weekend intensity factor (Table 4 does not show this factor, which multiplies the whole day’s volume by 0.4 on Saturdays and Sundays), so each day already carries the right pattern, and running seven separate days avoids having to decide what an idle boat should do overnight, when no demand exists in the model between hours 0 and 6.

Conservation check. Every person the demand generator creates must be accounted for at the end of a run, either as served, as still waiting in a queue, or as still on board a boat that had not yet reached its destination when the run ended. This identity, generated equals served plus still waiting plus still aboard, was checked automatically on all three runs and held exactly in every case, which is a basic but necessary sanity check given that the environment tracks people individually rather than as aggregate counts.

Table 6: Aggregate results of the reference policy at the three scales.

Metric	Step 1	Step 2	Step 3
Percent served	100.0%	100.0%	99.6%
Mean waiting time (min)	17.4	10.9	10.0
Mean / maximum time in system (min)	27.2 / 54.0	20.5 / 53.6	19.8 / 57.1
Waiting time, 50th / 90th / 95th percentile (min)	15.6 / 33.4 / 33.5	9.4 / 21.6 / 26.3	9.1 / 21.4 / 25.1

Table 6 reports the headline numbers. All three runs serve essentially everyone; the small shortfall in Step 3 (23 people out of 5429, none of them actually lost, just still on board a boat when one of the seven daily windows closed) reflects the mechanical fact that seven independent one-day cutoffs give seven separate chances for a boat to be mid-trip exactly when the clock runs out, not a failure of the policy. Because nothing in the environment discards a slow case anymore (Section 5.3), the percentiles reported here are the real, uncensored distribution of how long a served person actually waited, including its tail, rather than a distribution that quietly excludes whoever would have taken longest.

Step 2, despite carrying 5.5 times the demand of Step 1 with only one additional boat, reports a shorter mean wait and shorter tail percentiles. This is not a contradiction: Step 1 concentrates its entire demand into three peak hours with no slack period for a backlog to clear, while Step 2 spans eighteen hours with alternating busy and quiet bands, giving the fleet repeated opportunities to catch up between peaks.

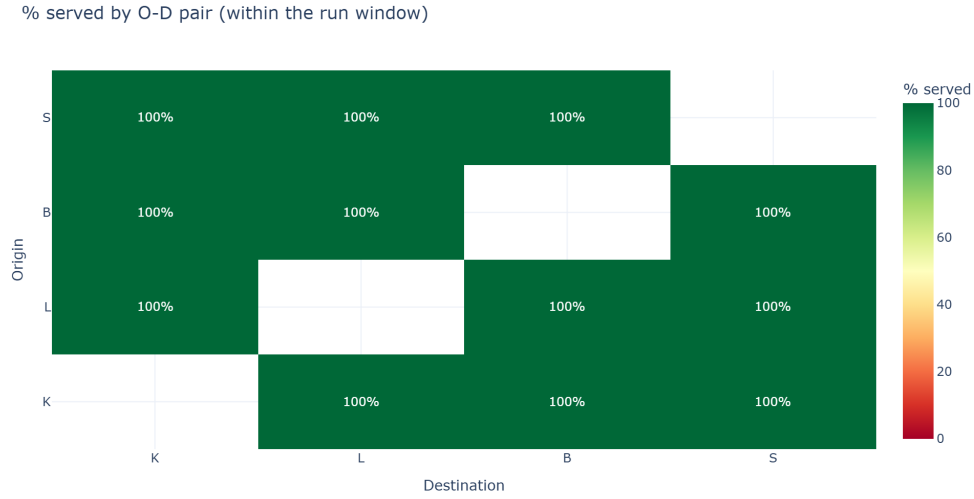


Figure 5: Percent of demand served per origin-destination pair, Step 2 (full day). K, L, B, and S denote Kleppstø, Laksevåg, Bryggen, and Sandviken.

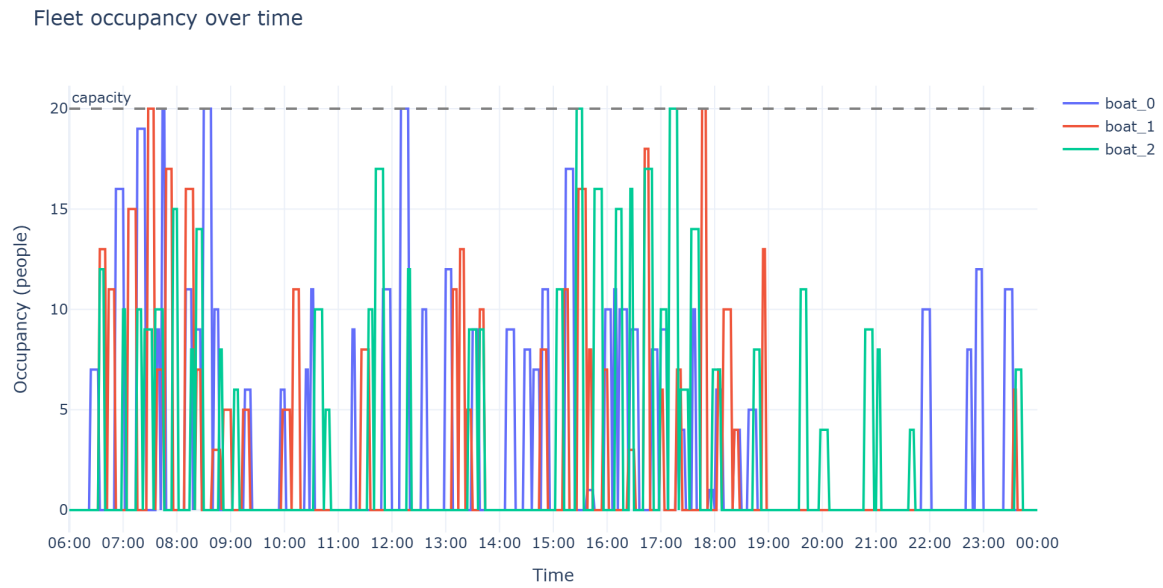


Figure 6: Occupancy of each boat over the full day, Step 2. Both boats spend a large share of the day near full capacity during the morning and afternoon commute bands.

People waiting over time

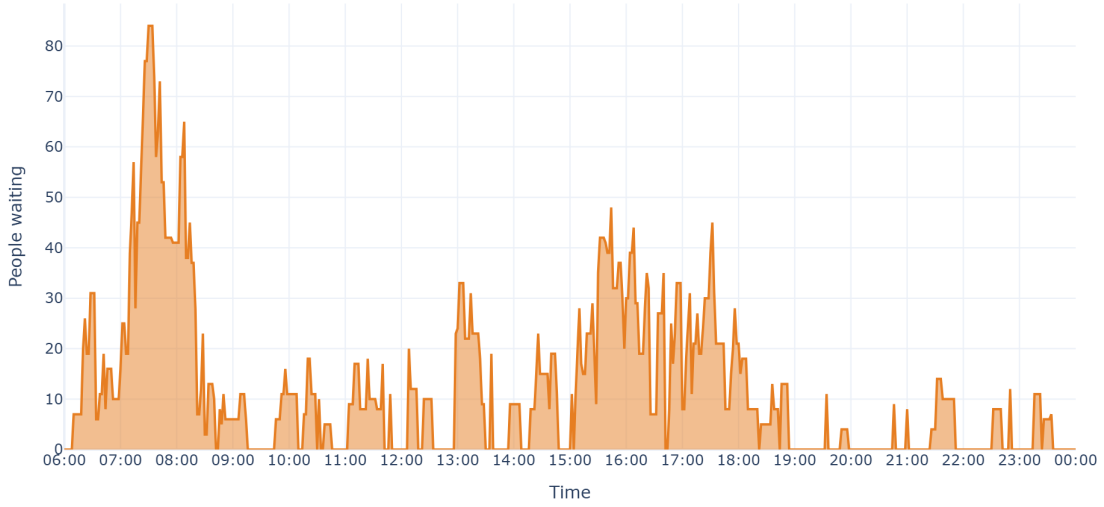


Figure 7: Number of people waiting, summed over all twelve pair queues, over the course of Step 2. The two peaks correspond to the morning and afternoon commute bands of Table 4.

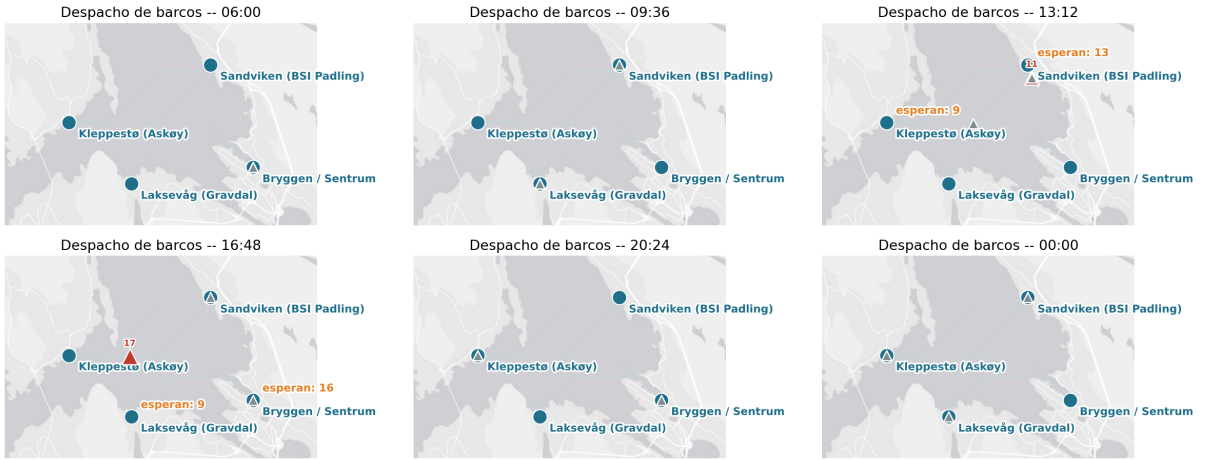


Figure 8: Six snapshots of the fleet over the course of Step 2, taken at even intervals from 06:00 to the following midnight. Orange labels show how many people are waiting at a stop at that instant; a full step-by-step, minute-by-minute view is available as an interactive, browser-based visualization produced by the same code, not reproducible as a static figure here.

Figure 5 breaks the aggregate percent-served figure down by origin-destination pair and confirms that every one of the twelve pairs reaches 100% in Step 2, not just the total. Figure 6 and Figure 7 show, respectively, how full each boat runs and how many people are waiting system-wide as the day unfolds; both track the two commute peaks of the demand model closely, which is the expected behavior of a policy that reacts to demand rather than anticipating it. Figure 8 gives a static sense of the fleet in motion; the underlying tool can also produce an animated, minute-by-minute version and a version that can be stepped through one decision at a time, both used during development to visually confirm the fleet-assignment fix of Section 6.2,

but neither reproduces meaningfully on a printed page.

6.4 Known limitations

Three limitations of this policy are worth stating explicitly rather than leaving implicit. First, the strict local-before-remote priority described above means a lightly connected pair can in principle wait indefinitely while a busier pair keeps the fleet occupied; this did not occur in the three runs reported here, but nothing in the rule itself prevents it, and a busier or more imbalanced instance could expose it. Second, fleet assignment (Section 6.2) prevents two boats from claiming the same destination in the same step, but it still assigns boats one at a time, in a fixed order, rather than searching over every possible joint assignment for the one that would serve the most people; it removes the worst failure mode without claiming to be optimal. Third, repositioning is myopic by construction, evaluating only the immediate move rather than the pickup and delivery as a planned pair, so a boat can occasionally reposition toward demand that a different, better-timed choice would have served faster. None of these are hidden costs: they are the price of a rule simple enough to state in a paragraph and verify by hand, which is exactly why it is a useful baseline rather than a proposed final design.

7 Reinforcement Learning Agent

The reference policy of Section 6 is a fixed rule with no parameters to tune; it cannot improve with experience, and it does not attempt to plan more than one move ahead. This section describes a first attempt to replace it with a policy that learns, reports how that first attempt initially failed in an informative way, and documents the diagnostic process that fixed it.

7.1 Method

The learning method used is proximal policy optimization, or PPO [Schulman et al., 2017], an on-policy, actor-critic reinforcement-learning algorithm. In broad terms, PPO alternates between two phases: it runs the current policy in the environment to collect a batch of experience (states visited, actions taken, rewards received), and it then updates the policy’s parameters to make the actions that led to above-average outcomes more likely and the actions that led to below-average outcomes less likely, using that same batch. What distinguishes PPO from earlier policy-gradient methods is a clipped objective function that keeps each update close to the policy that generated the data, which in practice makes training considerably more stable: an update that would change the policy too abruptly in a single step is capped rather than applied in full. This matters here because the environment is stochastic (a fresh demand realization every episode, Section 7.4) and the reward is comparatively sparse and delayed relative to the two-minute decision interval, both conditions under which more aggressive update rules tend to destabilize training.

PPO was chosen over value-based alternatives such as deep Q-networks specifically because of the action space described in Section 5.2: one of five discrete choices per boat, which for a fleet of n_b boats is a `MultiDiscrete` space of size 5^{n_b} , not a single flat discrete choice. The standard deep Q-network implementation in the software library used for training (Section 7.2) does not support `MultiDiscrete` action spaces natively, while PPO’s policy network naturally outputs one independent categorical distribution per boat and requires no adaptation to handle several simultaneous discrete decisions.

7.2 Implementation, observation, and action

The agent is implemented with Stable-Baselines3 [Raffin et al., 2021], a maintained, widely used reinforcement-learning library built on PyTorch, driving the same Gymnasium-standard

environment [Brockman et al., 2016] described in Section 5; no separate implementation of the environment exists for training, since the whole point of standardizing the environment’s interface in the first place was to let a learning library drive it without modification. The policy is Stable-Baselines3’s default `MlpPolicy`, a small feedforward network with two hidden layers of 64 units each and hyperbolic-tangent activations, shared as a common trunk between the policy head (which outputs the five-way action distribution for each boat independently) and the value head (which estimates expected future reward from the current state, used internally by PPO to judge whether a batch of experience was better or worse than expected).

The network’s input is exactly the fixed-size numeric vector described in Section 5’s discussion of state: one-hot node encodings and a normalized time-to-arrival for each boat, a normalized occupancy and free/busy flag for each boat, a normalized count of people waiting and a normalized oldest-waiting-time for each of the twelve pair queues, and sine/cosine encodings of the minute of day and day of week. For the two-boat, four-node instance used for training (Step 1 of Section 6.3), this vector has 50 entries. The network never sees the rich, dictionary-shaped state used for logging and inspection, and it never sees anything beyond this vector: no privileged information about, for instance, which specific person is waiting where, only the aggregated counts and times that the vector encodes.

7.3 Reward function

The reward returned by the environment at every step, introduced conceptually in Section 5, is the quantity PPO is trained to maximize in expectation; deferred there, its full definition is given here because it is only meaningful in the context of this training procedure. Let $\tau_{\text{tol}} = 12$ minutes be the comfort tolerance (the longest direct navigable trip, Table 1), and for every person still active in the system, either waiting or aboard a boat, let their surplus be the time they have spent in the system beyond that tolerance:

$$\text{surplus} = \max(0, \text{time in system} - \tau_{\text{tol}}). \quad (7)$$

Each active person’s discomfort penalty is a capped, quadratic function of that surplus,

$$\text{penalty} = \min\left(C, \left(\frac{\text{surplus}}{\sigma}\right)^2\right), \quad (8)$$

with normalizing width $\sigma = 18$ minutes and cap $C = 1.0$. The cap exists specifically because the environment no longer removes anyone from a queue (Section 5.3): without it, a single badly neglected person could accumulate an unbounded surplus and, through the square, dominate the entire step’s reward, drowning out the signal from everyone else. With the cap, a person’s penalty stops growing once they have waited 30 minutes past the tolerance (18 minutes of surplus, at which $(\sigma/\sigma)^2 = 1$), but it is never forgotten; it is charged at the maximum rate for as long as they remain unserved. Two further terms complete the reward: a small penalty for every boat under way in that step, and, only for training this agent, a positive term rewarding deliveries,

$$r = - \left[\sum_{\text{active persons}} \text{size} \cdot \text{penalty} + w_{\text{move}} \cdot (\text{boats under way}) \right] + w_{\text{deliver}} \cdot (\text{people delivered this step}), \quad (9)$$

where *size* is 1 for an individual person or the group’s headcount if the environment is configured to move whole groups at once (Section 4), and w_{move} , w_{deliver} are configuration values, not hard-coded constants. Table 7 gives a sense of scale: a person exactly at the tolerance contributes nothing, one who has waited 30 minutes past it contributes the full capped penalty of 1.0, and ten such people waiting at once contribute at most -10 to that step’s reward, never more, regardless of how much further any of them has to wait.

Table 7: Discomfort penalty by surplus, for one person of size 1.

Situation	Penalty
Waiting 12 minutes or less (within tolerance)	0
18 minutes total (6 minutes of surplus)	$(6/18)^2 \approx 0.11$
24 minutes total (12 minutes of surplus)	$(12/18)^2 \approx 0.44$
30 minutes total (18 minutes of surplus, the cap reached)	$(18/18)^2 = 1.00$
60 minutes total (48 minutes of surplus)	1.00 (uncapped value would be 7.1)

The two weights w_{move} and w_{deliver} deserve a note on where their values come from, since they are not the same for the reference-policy runs of Section 6.3 and for training this agent. In production, $w_{\text{move}} = 0.1$ and $w_{\text{deliver}} = 0$, meaning deliveries carry no direct reward and moving a boat is only lightly discouraged; this is the formula used to report reward figures throughout Section 6.3. Training this agent instead uses $w_{\text{move}} = 0$ and $w_{\text{deliver}} = 0.5$, an override applied only to the training and evaluation environments and never to the reference-policy runs. Section 7.5 explains why this override exists and how its value was chosen.

7.4 Fresh demand on every reset

Every run reported in Section 6.3 uses one fixed table of demand, generated once and reused for the whole run, which is the right choice for verifying that the environment and the reference policy behave correctly: the policy has no parameters to overfit, so there is nothing to gain from varying the demand across runs. Training a learning agent on a single fixed table of demand is a different matter and a real risk: with enough experience against exactly the same arrivals, at exactly the same simulated minutes, a large enough network can begin to memorize that particular realization rather than learn a policy that generalizes to the underlying arrival pattern it is drawn from. To avoid this, the training environment regenerates its demand table on every reset, drawing a new realization of the same gravity-and-Poisson process of Section 4 each time, so that every training episode is an independent Monte Carlo sample of the same demand pattern rather than a repeat of the previous one. A reset called with an explicit seed still reproduces that exact realization, which is what makes the fixed evaluation seeds of Section 7.6 comparable across runs.

7.5 Diagnosing and fixing a degenerate first training run

The agent was first trained for 100,000 timesteps, roughly 1115 episodes of the 90-step training instance, using Stable-Baselines3’s default hyperparameters and the production reward weights of Section 7.3. The result was a degenerate policy: it served only 27.6% of demand against 88.0% for the reference policy on the same evaluation seeds, moved the fleet less (79 boat movements against 155), and ran at a much lower average occupancy (1.15 against 3.50 passengers per boat). Rather than discarding this result, it was treated as a diagnostic starting point and investigated systematically, one change at a time, each verified before moving on to the next.

Step 1: a minimal sanity check. Before touching the reward or the training hyperparameters, the same untouched recipe that produced the degenerate policy above was tried on a much smaller, purpose-built instance: a single boat serving only two of the four nodes, over the same 90-step morning window. If a learning method cannot learn anything useful on a problem this small, the most likely explanation is a wiring bug in the environment itself, not a matter of scale or of reward shaping. On this toy instance the trained agent served *none* of the demand, against 67.1% for the reference policy on the same seed, even though the environment’s own contract checker (Stable-Baselines3’s `check_env`) reported no issues and the reward function

had already been confirmed, in isolation, to respond correctly to deliveries. This ruled out a structural bug and pointed toward the scale or shape of the training signal instead.

Step 2: observation and reward normalization. The single largest change came from wrapping the environment in `VecNormalize`, a standard Stable-Baselines3 utility that maintains running mean and variance statistics and uses them to rescale both the observation vector and the reward before they reach the network. Without it, the roughly 50-dimensional observation vector mixes one-hot indicators (naturally in $\{0, 1\}$) with counts and normalized times of quite different typical magnitudes, and the un-rescaled reward can swing by tens of units between quiet and busy steps; both are known sources of instability for gradient-based training. Adding normalization alone, with nothing else changed, took the toy instance from serving no one to matching the reference policy’s 67.1% exactly, with a better raw reward besides. This single change accounts for most of the improvement documented in this section.

Step 3: a positive term for deliveries. With normalization already in place, a positive reward term for each person delivered (Section 7.3, weight w_{deliver}) was added, so that the training signal has an explicit success case rather than only degrees of failure. On the toy instance this did not move the percentage served, which appeared to already be at that instance’s ceiling given a single boat, but it produced a small, consistent improvement in raw reward.

Step 4: a temporary reduction of the movement penalty. The movement penalty w_{move} was set to zero for training, again only for training, so that the agent is not discouraged from moving the fleet while it is still learning when moving is worthwhile. This is flagged explicitly as a temporary choice for this diagnostic phase, not a claim that movement should be free in a deployed policy.

Step 5: exploration and update frequency. PPO’s entropy coefficient, which rewards a more exploratory (less deterministic) policy during training and defaults to zero in Stable-Baselines3, was raised to 0.01, and the number of environment steps collected between policy updates was reduced from the default of 2048 to 512, so that the policy is updated roughly every five or six episodes of the 90-step training instance rather than waiting for about twenty two.

Step 6: confirming training and evaluation use the same instance. A mismatch between the instance an agent is trained on and the instance it is evaluated on was considered as a candidate explanation and checked directly, not merely assumed. Both the training and the evaluation code read the same configuration key for which instance to use, and an explicit check, comparing the shape of the observation vector the loaded model expects against the shape the evaluation environment produces, was added to the evaluation procedure so that a future mismatch of this kind would stop the run with a clear error rather than silently producing an incomparable result. No mismatch was found in this project; the check is retained as a safeguard.

Table 8: Diagnostic steps on the toy instance (one boat, two nodes) and their effect, evaluated against the reference policy on the same seed.

Configuration	Percent served	Reward (agent)	Reward (base)
Original recipe (Step 1 above)	0.0%	-13,218.9	-3694.0
+ Observation/reward normalization	67.1%	-3521.4	-3694.0
+ Delivery reward	67.1%	-3416.4	-3589.0
+ Movement penalty removed (training only)	67.1%	-3408.9	-3581.5
+ Higher entropy, more frequent updates	67.1%	-3408.9	-3581.5

A bug found while building the evaluation procedure. A second, unrelated implementation error surfaced only once the fixes above were carried over to the full-scale instance and evaluated: the first version of the evaluation code drove the environment through Stable-Baselines3’s vectorized-environment wrapper in order to reuse its normalization statistics, and that wrapper, by design, automatically resets an environment the instant an episode ends, inside the same function call that reports the episode as finished. This silently erased the very history (who was served, at what time) that the evaluation code needed to read immediately afterward, and it was caught only because the conservation check of Section 6.3 was run on the evaluation output too and failed outright, reporting zero people served against a nonzero number generated. The fix was to step the raw environment directly during evaluation and use the saved normalization statistics only to rescale the observation before it is shown to the network, never routing the actual simulation through the auto-resetting wrapper. This is reported here as a concrete illustration of why the conservation check of Section 6.3 is applied everywhere in this project, not only to the reference-policy runs it was originally written for.

7.6 Training run and comparison against the reference policy

With the combination of changes from Section 7.5 validated on the toy instance, the agent was retrained on the real two-boat, four-node training instance (Step 1 of Section 6.3) for 150,000 timesteps, about 1666 episodes; a longer run of 300,000 timesteps was attempted first but did not complete in reasonable time on the machine available for this project (an observed throughput of roughly 73 simulated steps per second on this instance), so 150,000 was used as a practical compromise, with more training time flagged below as the most direct next step.

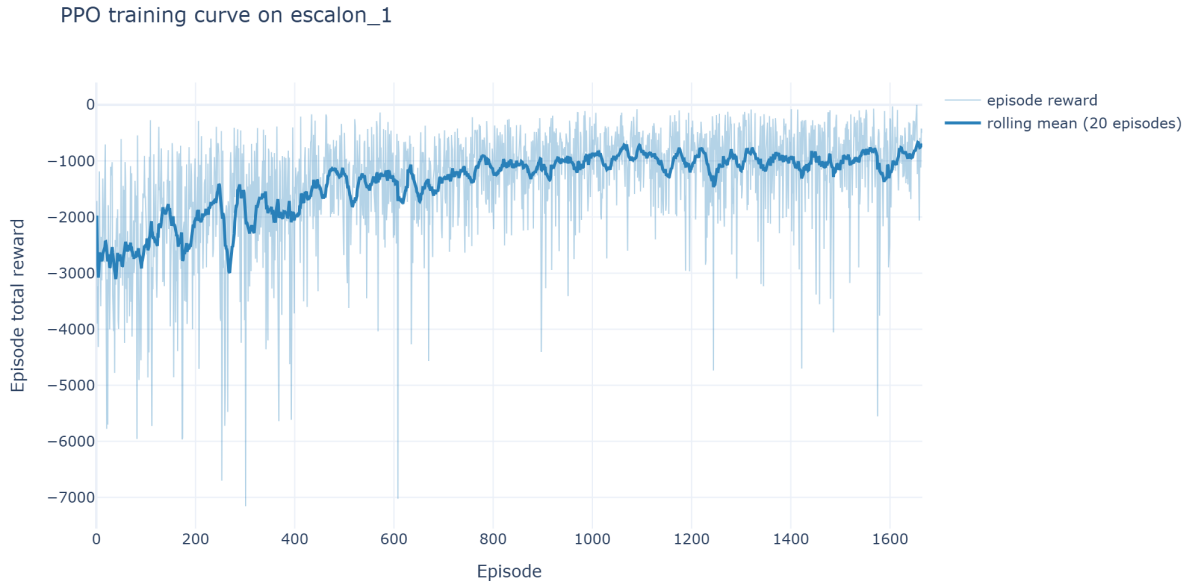


Figure 9: Training curve: total reward per episode (light) and its 20-episode rolling mean (dark), over 1666 training episodes.

Figure 9 shows the resulting curve. The rolling mean rises from about -2439 over the first 20 episodes to about -691 over the last 20, a substantially larger improvement than the first, unfixed attempt managed, and it had clearly not flattened out when training stopped, meaning the agent was still improving at the point the run was cut off for time.

The trained agent was then compared against the reference policy on five fixed evaluation seeds, deliberately outside the range of seeds used during training, with the same demand realization given to both policies for each seed so the comparison is on equal terms. Both policies were run through the same reward formula, including the training-time weights of Section 7.3, so that the reward figures reported below are directly comparable to each other, though not to the production reward figures of Section 6.3.

Table 9: Agent versus reference policy, averaged over five held-out evaluation seeds.

Metric	PPO agent	Reference policy
Percent served	77.2%	88.0%
Mean waiting time (min)	20.5	16.4
Mean / maximum time in system (min)	30.4 / 117.1	26.1 / 69.7
Boat movements (5 episodes, total)	177	155
Mean occupancy	2.97	3.50
Total reward (5 episodes)	-4807.0	-2009.3

Agent vs. base policy: percent served by O-D pair

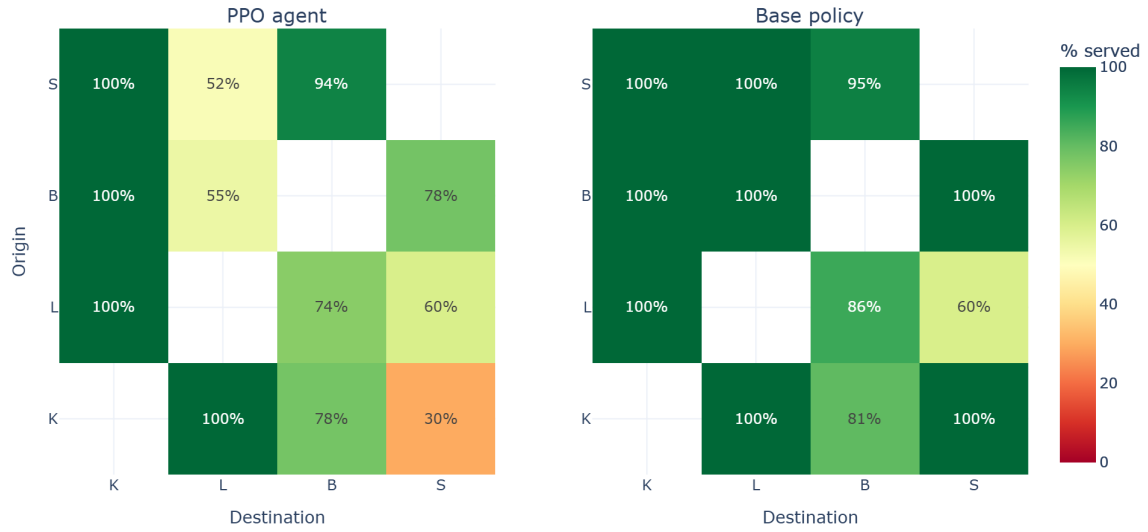


Figure 10: Percent served by origin-destination pair, agent (left) against reference policy (right), same five evaluation seeds.

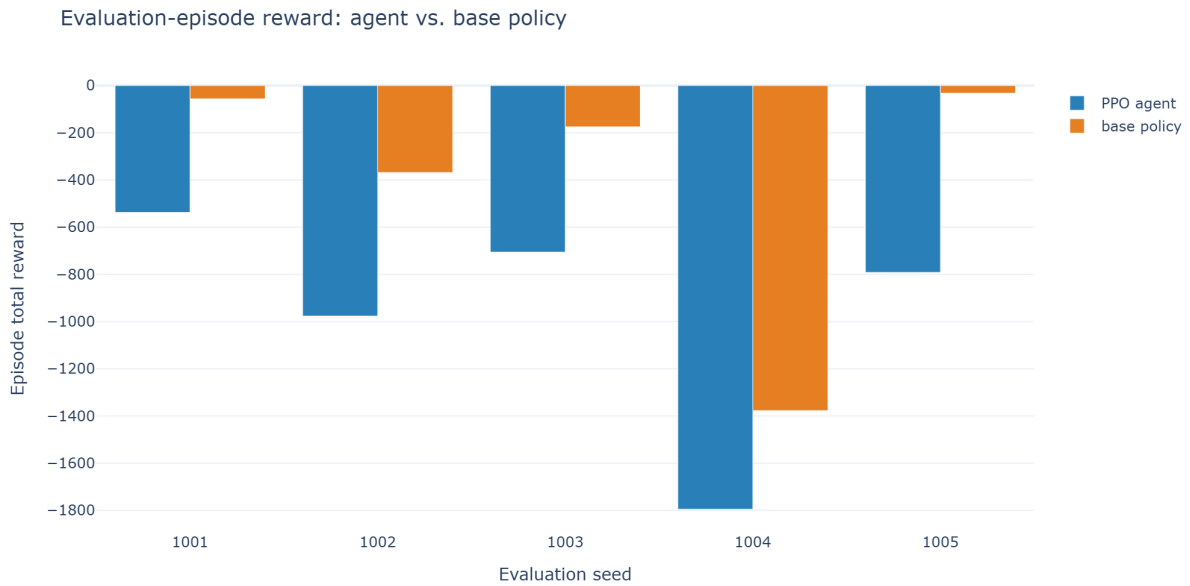


Figure 11: Total reward per evaluation episode, agent against reference policy, one bar pair per evaluation seed. The reference policy is ahead on every single seed, not only on average.

Table 9 and Figures 10-11 tell a consistent story: the diagnostic work of Section 7.5 closed most of the gap from the original, degenerate attempt, but the agent has not yet matched the reference policy. It serves 77.2% of demand against 88.0%, and its worst pairs (30% for Kleppstø to Sandviken, 52% for Sandviken to Laksevåg) are considerably weaker than the reference

policy’s weakest pair (60%). Its maximum time in system, 117 minutes, is also markedly worse than the reference policy’s 69.7 minutes, indicating that at least a few people are left waiting substantially longer under the agent than under the fixed rule.

Is the agent simply refusing to move the fleet. This was a natural hypothesis, given the earlier, fully degenerate attempt, and it was checked directly rather than assumed either way, by counting, from the logged record of every decision each boat made, how often each policy chose to wait rather than travel. The finding contradicts the hypothesis: the agent waits in only 13.0% of its decisions (31 out of 238, and one of its two boats never waits at all across all five evaluation episodes), against 47.5% (140 out of 295) for the reference policy, and it accumulates more total time under way (1334 minutes against 1224). The agent moves the fleet *more*, not less, than the reference policy, and still serves fewer people; the shortfall is better explained by some of that additional movement being poorly directed, for instance a repositioning trip that arrives too late or is not the one the situation most needed, than by any reluctance to dispatch boats at all.

7.7 Discussion and next step

Taken together, the diagnostic process of Section 7.5 and the results of Section 7.6 support a specific, narrow conclusion rather than a general one about whether reinforcement learning can work here. The first training attempt failed for identifiable, fixable reasons, chiefly the absence of observation and reward normalization, not because of a flaw in the environment or the reward design; once fixed, the same recipe reliably reaches the reference policy’s level on a small instance and closes most, though not all, of the gap on the real training instance. The training curve of Figure 9 had not leveled off when the run was stopped, which is the most direct evidence available that the remaining gap is, at least in part, simply a matter of training budget rather than a ceiling the method cannot get past. The most direct next step is therefore to train substantially longer before drawing any conclusion about whether proximal policy optimization can match, and eventually exceed, this reference policy on this problem; the goal for this stage of the project was for the agent to reach parity with the fixed rule, not to surpass it, and that goal has not yet been met, which is reported here plainly rather than understated.

References

- Caroline Braathen, Julio C. Goez, and Mario Guajardo. Autonomous ferries in light of labor regulations – a passenger perspective. *Maritime Transport Research*, 7:100115, 2024. doi: 10.1016/j.martra.2024.100115.
- Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym. *arXiv preprint arXiv:1606.01720*, 2016.
- Yong Gu and Stein W. Wallace. Operational benefits of autonomous vessels in logistics – a case of autonomous water-taxis in Bergen. *Transportation Research Part E: Logistics and Transportation Review*, 154:102456, 2021. doi: 10.1016/j.tre.2021.102456.
- Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.
- Gabriel E. Sánchez-Martínez, Nigel H. M. Wilson, and Haris N. Koutsopoulos. Schedule-free high-frequency transit operations. *Public Transport*, 9:285–305, 2016. doi: 10.1007/s12469-016-0129-8.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.